

Chapter Overview

- 16.1 Introduction
- 16.2 Open Database Connectivity
- 16.3 OLE DB
- 16.4 ActiveX Data Object
- 16.5 ADO Data Control
 - 16.5.1 Properties of ADO Data Control
- 16.6 ADO Object Model
- 16.7 Connection Object
 - 16.7.1 Properties of Connection Object
 - 16.7.2 Collections of Connection Object
- 16.8 Recordset Object
 - 16.8.1 Properties of Recordset Object
 - 16.8.2 Collections of Recordset Object
 - 16.8.3 Methods of Recordset Object
- 16.9 Command Object
 - 16.9.1 Properties of Command Object
 - 16.9.2 Collections of Command Object
- 16.10 Microsoft FlexGrid Control
 - 16.10.1 Adding MSFlexGrid Control In Application
 - 16.10.2 Important Properties of MSFlexGrid
 - 16.10.3 Using MSFlexGrid with Database
- 16.11 Master-Detail Tables
 - 16.11.1 Creating Master-Detail Tables in Microsoft Access
 - 16.11.2 Displaying Data from Master-Detail Tables

Short Questions
Multiple Choices
Fill in the Blanks
True/False

16.1 Introduction

Each DBMS stores data in a particular format. For example, Microsoft Access stores data in the 'mdb' file while FoxPro uses the 'dbf' format etc. Traditionally, a database application was developed for a particular type of database. If you change the DBMS, you also need to rewrite the application to handle data in a new format.

A method was required for accessing databases such that the application remains unchanged even the databases is changed. This is known as **open database connectivity**. It is possible to build applications that are able to communicate with different databases. This is possible if the application gives a command in a particular manner. This command is then translated such that it is understood by the DBMS. The user can communicate with different databases by using different translators between the application and the DBMS.



16.2 Open Database Connectivity

ODBC uses a standard method for communication between the application and the DBMS. This is accomplished by defining an Application Programming Interface (API). An API in programming terms means a set of functions that perform a particular task.

The application will call an API function to specify a particular task. The API call is passed on to the translator. The translator issues command in the language of DBMS to specify the requested action. The translator is referred to as the **ODBC Driver**. It is the driver that implements the various API such that they are understood by the DBMS. Thus, there is a driver for each DBMS to be accessed.

The **ODBC Driver** accepts the command from the application and converts it to a format understood by the target DBMS. In addition, the **ODBC Driver** also receives the result of the command execution from the DBMS and passes it back to the application.

16.3 OLE DB

ODBC defined API that could be used to access any data but it did not support the object orientation approach. **OLE DB** is a database architecture that enables applications to have uniform access to data stored in diverse information sources, regardless of the location or type. **OLE DB** defines a set of COM (Component Object Model) interfaces that enable a uniform access to data stored in diverse DBMS.

16.4 ActiveX Data Object

ActiveX Data Object (ADO) is a Microsoft technology that is used to access and manipulate databases. It works with OLE DB for providing interfaces between programs and data sources. This is the latest technology for connecting to different types of data sources like Microsoft Access, Oracle, SQL Server and many other data sources.

Data control is not a very flexible tool as it can work only with limited kinds of data. It must work strictly in the Visual Basic environment. To overcome these limitations, we can use a much more powerful data control in VB known as ADO control. ADO is ActiveX-based and it can work in different platforms and different programming languages. It can access many different kinds of data such as data displayed in the Internet browsers, email text and even graphics other than the usual relational and non-relational database information.

The common way to access a database by using ADO is to:

1. Create an ADO Connection to a database.
2. Open the database Connection.
3. Create an ADO Recordset.
4. Open the Recordset.
5. Extract the data you need from the Recordset.
6. Close the Recordset.
7. Close the Connection.

16.5 ADO Data Control

The ADO Data control is similar to Data Control in use. You need to add it in the toolbox from Components dialog box.

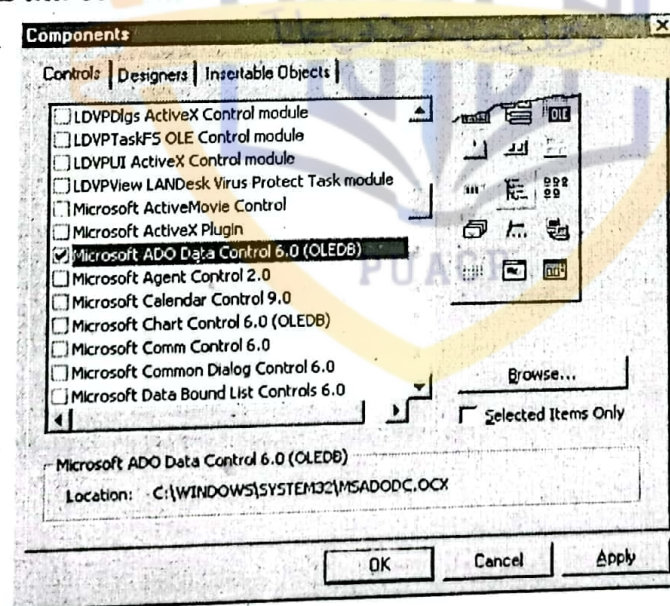
16.5.1 Properties of ADO Data Control

Some important properties of ADO Data control are as follows:

- **ConnectionString:** This property is used specify a string that is used to develop a connection with the database.
- **LockType:** This property is used to specify the type of lock on the records.
- **RecordSource:** This property is used to specify the table to access data.

Practical 16.1

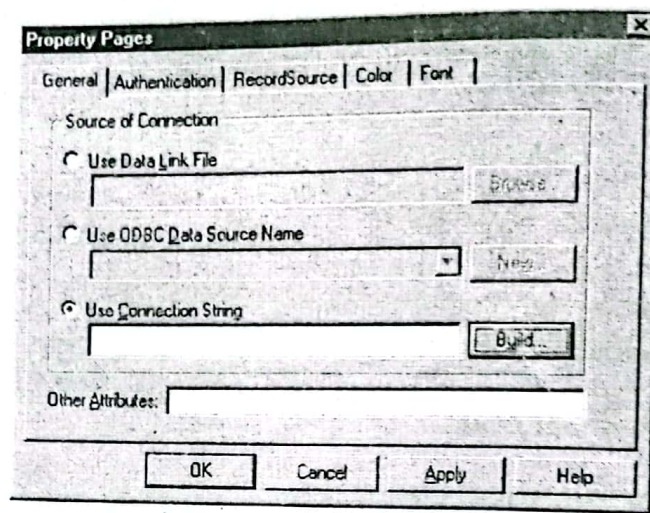
1. Start a new Standard EXE project.
2. Select Components... from Project menu. A dialog box will appear.
3. Select Microsoft ADO Data Control 6.0 (OLEDB) from the dialog box and click OK. ADO Data control will be added in the toolbox.



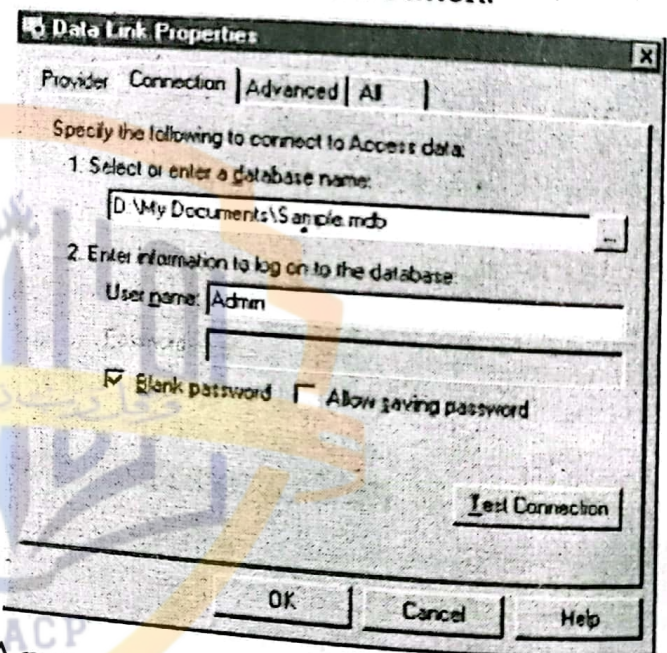
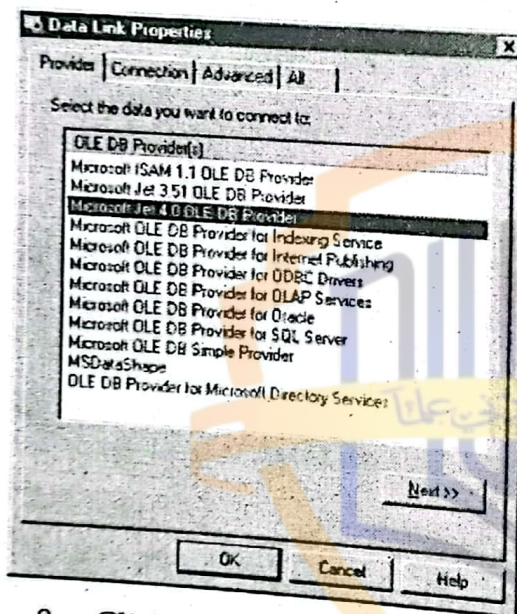
4. Double on the ADO Data Control in the toolbox. It will be added in the form.



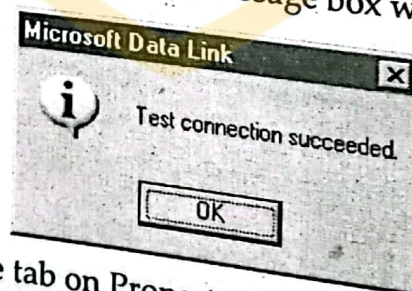
5. Right click on ADO Data control and select ADODC Properties. The Property Pages window will appear.



6. Select Use Connection String option and click on Build... button. The Data Link Properties window will appear.
7. Select Microsoft Jet 4.0 OLE DB Provider and click Next button.



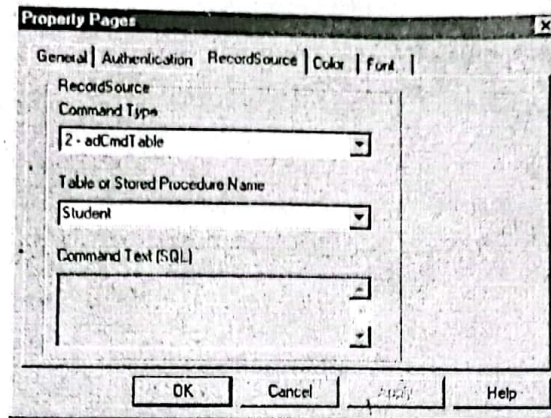
8. Click Test Connection button. A message box will appear.



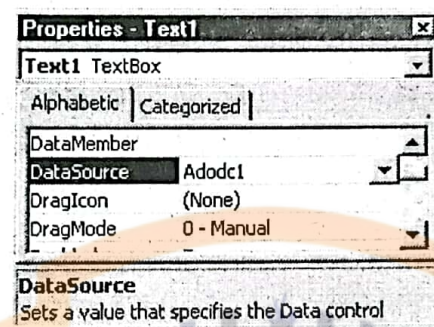
9. Select RecordSource tab on Property Page.
10. Select adCmdTable from Command Type list.

Type	Description
adCmdUnknown	Unknown command. This is the default value.
adCmdText	Used to specify SQL command in the Command Text box.
adCmdTable	Displays a list of tables in the database we are connected to.
adCmdStoredProc	Displays all the available queries or stored procedures.

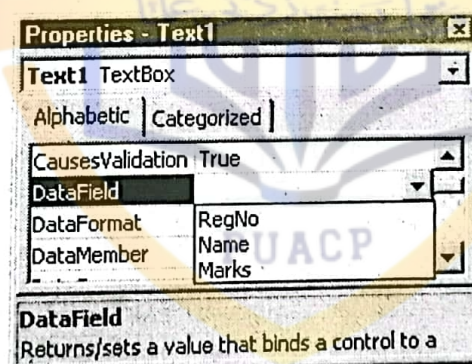
11. Select Student from Table list.



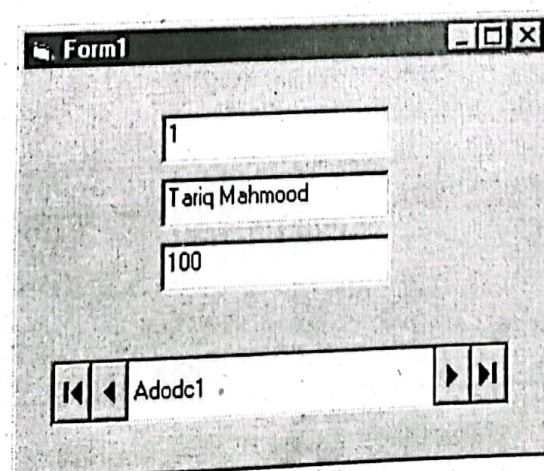
12. Click OK.
13. Place three textboxes on form and set their DataSource property to Adodc1.



14. Set DataField property of Text1 to RegNo.
15. Set DataField property of Text2 to Name.
16. Set DataField property of Text3 to Marks.



17. Run the project. The form will look like:



16.6 ADO Object Model

The ADO object model consists of the following objects:

Connection Object

This object represents an open connection. It is used to establish a connection between data source and Visual Basic application. You can access and manipulate database through this object.

Command Object

This object defines a specific command you wish to execute against data.

Recordset Object

This object represents a set of records from a table, command object, or SQL query. It can be created independently, without any underlying Connection object.

Using ADO Objects in Visual Basic

In order to use ADO objects in Visual Basic, you need to create a reference. From the Projects menu, choose References, and then select **Microsoft ActiveX Data Objects Library**.

16.7 Connection Object

The ADO Connection object is used to create an open connection to a data source to access and manipulate a database. This object is not essential for manipulating a database. You can also connect to a database by using Command or Recordset object. However, such a connection is only good for a particular single query. If you desire to access a data source multiple times, it is far more efficient to establish a connection using the Connection object.

16.7.1 Properties of Connection Object

Some important properties of Connection object are as follows:

Attributes

It sets or returns a long value defining the characteristics of a Connection object.

Syntax: ConnectionObject.Attributes

CommandTimeout Property

It sets the number of seconds to wait while attempting an Execute method. If the attempt is not successful within specified time, an error message is generated.

Syntax: ConnectionObject.CommandTimeout

ConnectionString Property

It sets or returns string value that contains details to create a connection to data source.

Syntax: ConnectionObject.ConnectionString

Property	Description
Provider	Specifies the name of the provider used for the connection.
Data Source	Specifies the name of a data source for the connection.
User ID	Specifies the user name to open a connection.
Password	Specifies the password to open a connection.

Table: Parameters of ConnectionString

ConnectionTimeout Property

It sets the number of seconds to wait while attempting to create a connection. If connection is not established within specified time, an error message is generated.

Syntax: ConnectionObject.ConnectionTimeout

DefaultDatabase Property

It sets or returns a string value that is the default name of the database available from the provider for a Connection object.

Syntax: ConnectionObject.DefaultDatabase

Provider Property

It sets or returns the string value that is the provider name.

Syntax: ConnectionObject.Provider

State Property

It returns a value describing if the connection is open or closed.

Syntax: ConnectionObject.State

Version Property

It returns a string value that is the ADO version number.

Syntax: ConnectionObject.Version

16.7.2 Collections of Connection Object

Some important collections of Connection object are as follows:

Errors Collection

It is a collection of Error objects generated by the failure of a connection operation.

Properties Collection

This is a collection of Property objects.

16.7.3 Methods of Connection Object

Some important methods of Connection object are as follows:

BeginTrans Method

It begins a new transaction and returns a long value indicating the number of nested transactions.

Syntax: ConnectionObject.BeginTrans

Close Method

It closes a connection.

Syntax: ConnectionObject.Close

CommitTrans Method

It saves any changes and ends the current transaction. It can also be set to automatically start a new transaction.

Syntax: ConnectionObject.CommitTrans

Execute Method

It executes the query, SQL statement, stored procedure, or provider-specific text.

Syntax: ConnectionObject.Execute (CommandText, RecordsAffected, Options)

Open Method

It opens a physical connection to a data source.

Syntax: ConnectionObject.Open (ConnectionString, UserID, Password, Options)

Parameters	Description
ConnectionString	Optional. A string containing the Connection information.
UserID	Optional. User name required to establish the connection.
Password	Optional. Password required when establishing the connection.

Table: Parameters of Open method

RollbackTrans Method

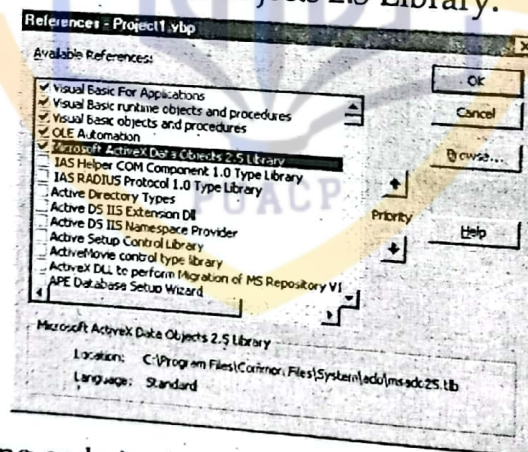
It cancels any changes that have occurred during the current transaction and then ends the transaction. It can also be set to automatically start a new transaction.

Syntax: ConnectionObject.RollbackTrans

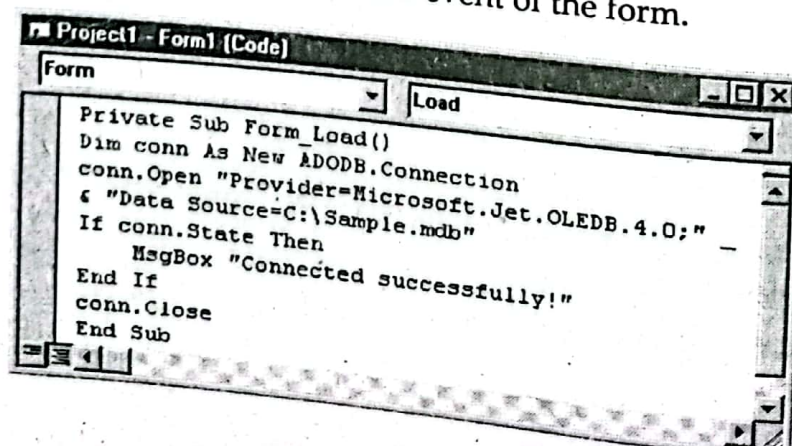
Practical 16.2

The following practical demonstrates how to connect to a database using Connection object of ADO.

1. Start a new Standard EXE project.
2. Select **References** from **Project** menu. The **References** dialog box will appear.
3. Select **Microsoft ActiveX Data Objects 2.5 Library**.



4. Add the following code in the Load event of the form.



5. Run the project. "Connected successfully" message will appear if the connection is established successfully.

How It Works?

The first line in the above example declares an object variable of type Connection. The second statement uses the built-in method to open the connection. Different options of Open method are as follows:

- **Provider** – Name of the provider. We are using Microsoft.Jet.OLEDB.4.0.
- **Data Source** – Name of and path of the data source for establishing connection.

If the connection establishes successfully, State property contains 1 value. So the message is displayed indicating that the connection has been established.

16.8 Recordset Object

The ADO Recordset object is used to store the data retrieved from a database. This object consists of records and fields. The Recordset object is considered to be the heart of ADO. We can retrieve and modify data by using this object.

The Recordset object contains many properties, collections, methods, and events that are used to manipulate the retrieved data.

16.8.1 Properties of Recordset Object

Some important properties of Recordset object are as follows:

ActiveCommand Property

It returns a variant that is the Command object associated with the Recordset object.

Syntax: RecordsetObject.ActiveCommand

ActiveConnection Property

It sets or returns a variant defining the Connection object to which the Recordset belongs, or returns a string value if there is no connection.

Syntax: RecordsetObject.ActiveConnection

BOF Property

It returns a Boolean value indicating if the current record position is before the first record or not.

Syntax: RecordsetObject.BOF

CursorType Property

It sets or returns the type of cursor being used.

Syntax: RecordsetObject.CursorType

Cursor Type	ADO Constant	Description
Forward- Only	adOpenForwardOnly	Behaves like a dynamic cursor but allows to scroll only forward through records.
Keyset	adOpenKeyset	Prevents from seeing the records other users add, and prevents access to records other users delete.
Dynamic	adOpenDynamic	Allows additions, changes, deletions and allows all types of navigation of Recordset.
Static	adOpenStatic	Allows all types of movement through Recordset.

EOF Property

It returns a Boolean value indicating if the current record position is after the last record or not.

Syntax: RecordsetObject.EOF

Filter Property

It sets or returns a value that is used to filter data.

Syntax: RecordsetObject.Filter

LockType Property

It sets or returns the type of locks that are in effect while editing records.

Syntax: RecordsetObject.LockType

Lock Type	Description
AdLockReadOnly	(Default) Read-only. Data cannot be altered.
AdLockPessimistic	The page containing the record is locked immediately on editing.
AdLockOptimistic	The page containing the record is locked only when the records are updated.
AdLockBatchOptimistic	Uses the BatchUpdate method to write the entire batch of rows to the Recordset.

Table: Lock Type options

MaxRecords Property

It sets or returns the maximum number of records that can be returned to a Recordset object as the result of a query.

Syntax: RecordsetObject.MaxRecords

RecordCount Property

It returns the number of records in a Recordset object.

Syntax: RecordsetObject.RecordCount

Sort Property

It sets or returns a string value that is the names of fields in the Recordset to sort.

Syntax: RecordsetObject.Sort

State Property

It returns a value describing if the Recordset object is open, closed, or in the process of connecting, executing, or retrieving.

Syntax: RecordsetObject.State

16.8.2 Collections of Recordset Object

Some important collections of Recordset object are as follows:

Fields Collection

A collection containing all of the Field objects associated with a Recordset object.

Properties Collection

It is a collection of Property objects.

16.8.3 Methods of Recordset Object

Some important methods of Recordset object are as follows:

AddNew Method

It is used to create a new record.

Syntax: RecordsetObject.AddNew (FieldList, Values)

Close Method

It closes a Recordset object.

Syntax: RecordsetObject.Close

Delete Method

It deletes the current record, a group of records, or all records.

Syntax: RecordsetObject.Delete (AffectRecords)

Find Method

It searches for a row in a Recordset that matches the given criteria.

Syntax: RecordsetObject.Find (Criteria, SkipRows, SearchDirection, Start)

Move Method

It moves the position of the current record pointer.

Syntax: RecordsetObject.Move (NumRecords, Start)

MoveFirst Method

It moves the position of the current record pointer to the first record.

Syntax: RecordsetObject.MoveFirst

MoveLast Method

It moves the position of the current record pointer to the last record.

Syntax: RecordsetObject.MoveLast

MoveNext Method

It moves the position of the current record pointer forward to the next record.

Syntax: RecordsetObject.MoveNext

MovePrevious Method

It moves the position of the current record pointer back to the previous record.

Syntax: RecordsetObject.MovePrevious

Open Method

When used on a Recordset object, this opens a cursor that is used to navigate through records.

Syntax: RecordsetObject.Open (Source, ActiveConnection, CursorType, LockType, Options)

Requery Method

It is used to update (refresh) the data in a Recordset object. This is essentially equivalent to a Close followed by an Open.

Syntax: RecordsetObject.Requery Options

Seek Method

It uses the index of a Recordset to locate a specified row.

Syntax: RecordsetObject.Seek KeyValues, SeekOption

Update Method

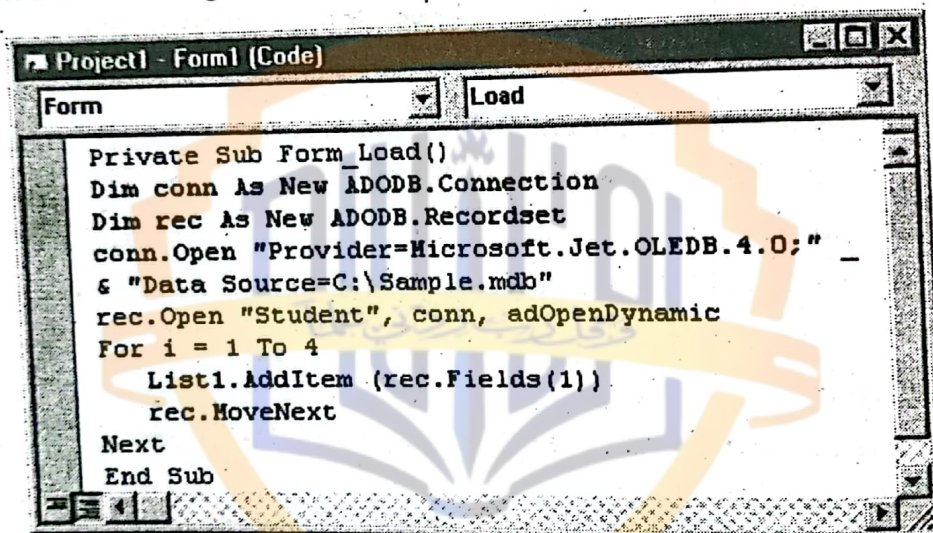
Used to save any changes made to the current row of a Recordset object.

Syntax: RecordsetObject.Update (Fields, Values)

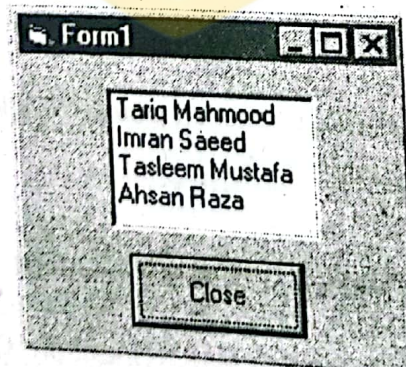
Practical 16.3

The following practical demonstrates how to retrieve data from database in Recordset.

1. Start a new Standard EXE project.
2. Place a List box and a button on the form.
3. Set the Caption of the button to "Close" and write "End" in its click event.
4. Select References from Project menu. The References dialog box will appear.
5. Select Microsoft ActiveX Data Objects 2.5 Library.
6. Add the following code in the Load event of the form.



7. Run the project. The list box will be filled with names from database table as follows:



How It Works?

The above example uses a Recordset to retrieve data from Sample database. The first argument of Open method rec is the name of table from which records are retrieved. The second argument specifies Connection object that refers to the database that contains Student table. The third argument is the type of Recordset. The For...Next loop adds second column of all records in a list box. The statement rec.Fields(1) refers to the index of the Recordset.

16.9 Command Object

Command object is used to submit and execute a query against a database. A query can perform different actions such as adding, creating, deleting, retrieving or updating databases. If the query is used to retrieve data, then the data will be returned as a Recordset object. The retrieved data can be stored in a Recordset object. An important feature of Command object is the ability to use stored queries and procedures that accept parameters. It requires access to Parameter collection

16.9.1 Properties of Command Object

Some important properties of Command object are as follows:

ActiveConnection

It sets or returns a variant value defining the Connection object to which the Command object belongs, or returns a string value if there is no connection.

Syntax: CommandObject.ActiveConnection

CommandText

It sets or returns the text of a command statement to be applied against data provider.

Syntax: CommandObject.CommandText

Name

It sets or returns the string name of the Command object.

Syntax: CommandObject.Name

State

It returns a long value describing if the Command object is open, closed or in the process of connecting, executing or retrieving.

Syntax: CommandObject.State

16.9.2 Collections of Command Object

Some important collections of Command object are as follows:

Parameters Collection

A collection of Parameter objects that allow the passing of parameters into stored queries and stored procedures.

Properties Collection

A collection of Property objects. Each Property object contains a provider-specific property.

16.9.3 Methods of Command Object

Some important methods of Command object are as follows:

Cancel

It cancels the execution of a pending Execute call.

Syntax: CommandObject.Cancel

Execute

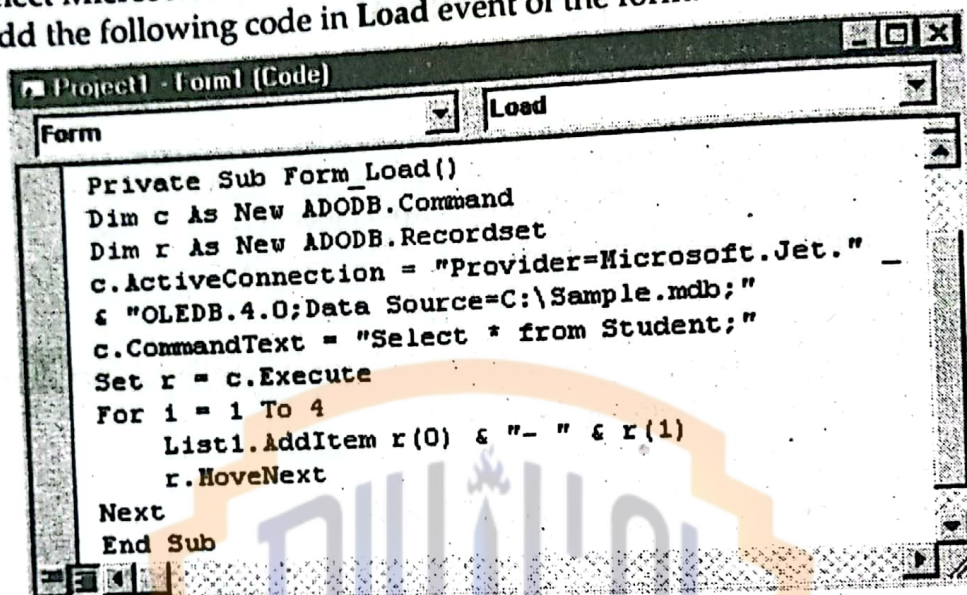
It executes the query, SQL statement or stored procedure.

Syntax: CommandObject.Execute (NumberRecords, Parameters, Options)

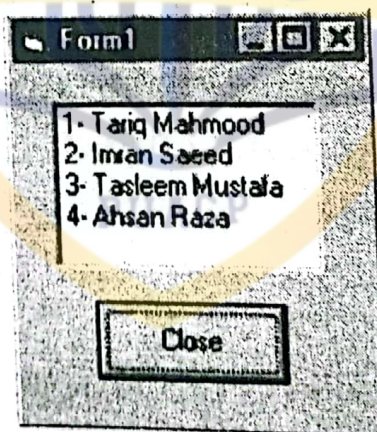
Practical 16.4

The following practical demonstrates how can you access data using Command object of ADO.

1. Start a new Standard EXE project.
2. Place a List box and a button on the form.
3. Set the Caption of the button to "Close" and write "End" in its click event.
4. Select References from Project menu. The References dialog box will appear.
5. Select Microsoft ActiveX Data Objects 2.5 Library.
6. Add the following code in Load event of the form.



7. Run the project. The list box will be filled with names from database table:

**How It Works?**

The above example does not use Connection object for establishing connection with database. It uses Command object and Recordset objects. The information of connection is stored in ConnectionString property of the Command object. The CommandText property contains SQL statement for retrieving data from the Student table. The Execute method executes the SQL statement and returns the result from the table, which is stored in the Recordset r. The For...Next loop fills the list box with the first two fields of the table.

Manipulating Database with ADO

You can perform all function on databases by using ADO objects. You can add, delete, update, navigate and search records in database by writing code instead of ADO control.

Practical 16.5

1. Start a new Standard EXE project.
2. Design the form as follows:

3. Select References from Project menu. The References dialog box will appear.
4. Select Microsoft ActiveX Data Objects 2.5 Library.
5. Add the following code in the General Declaration section.

```

Dim conn As New ADODB.Connection
Dim rec As New ADODB.Recordset

```

6. Add the following code in the Load event of the form.

```

Private Sub Form_Load()
    conn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=C:\Sample.mdb"
    rec.Open "Student", conn, adOpenDynamic, adLockBatchOptimistic
    Set txtRegNo.DataSource = rec
    Set txtName.DataSource = rec
    Set txtMarks.DataSource = rec
    txtRegNo.DataField = ("RegNo")
    txtName.DataField = ("Name")
    txtMarks.DataField = ("Marks")
End Sub

```

7. Add the following code in Click event of cmdFirst:

```

Private Sub cmdFirst_Click()
    rec.MoveFirst
End Sub

```


8. Add the following code in Click event of cmdPrevious:

```
Project1 - Form1 (Code)
cmdPrevious Click
Private Sub cmdPrevious_Click()
    rec.MovePrevious
    If rec.BOF Then
        rec.MoveFirst
    End If
End Sub
```

9. Add the following code in Click even of cmdNext:

```
Project1 - Form1 (Code)
cmdNext Click
Private Sub cmdNext_Click()
    rec.MoveNext
    If rec.EOF Then
        rec.MoveLast
    End If
End Sub
```

10. Add the following code in Click event of cmdLast:

```
Project1 - Form1 (Code)
cmdLast Click
Private Sub cmdLast_Click()
    rec.MoveLast
End Sub
```

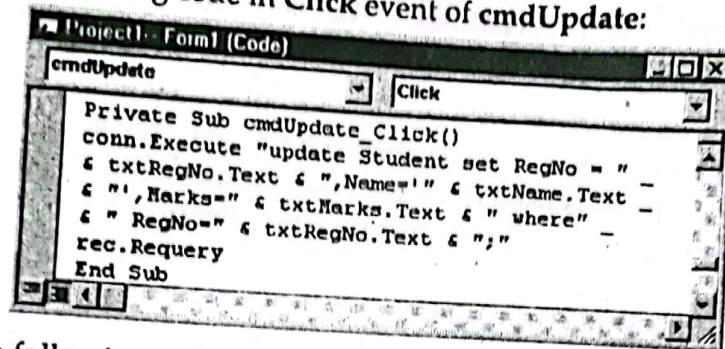
11. Add the following code in Click event of cmdAdd:

```
Project1 - Form1 (Code)
cmdAdd Click
Private Sub cmdAdd_Click()
    If txtRegNo = "" Or txtName = "" Or txtMarks = "" Then
        MsgBox "Enter complete data"
    Else
        conn.Execute "insert into Student values(" & txtRegNo.Text & "," & txtName.Text & "," & txtMarks.Text & ");"
        rec.MoveFirst
    End If
End Sub
```

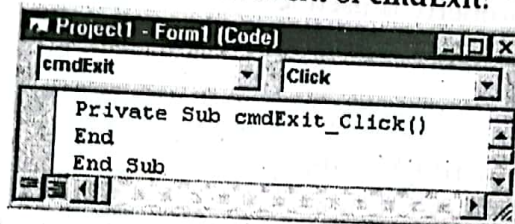
12. Add the following code in Click event of cmdDelete:

```
Project1 - Form1 (Code)
cmdDelete Click
Private Sub cmdDelete_Click()
    conn.Execute "delete from Student where RegNo = " & txtRegNo.Text
    rec.Requery
End Sub
```

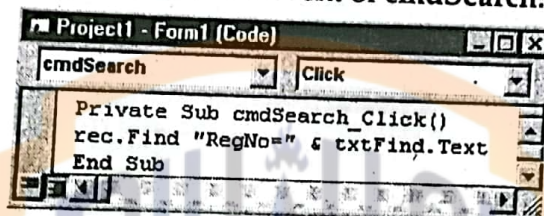

13. Add the following code in Click event of cmdUpdate:



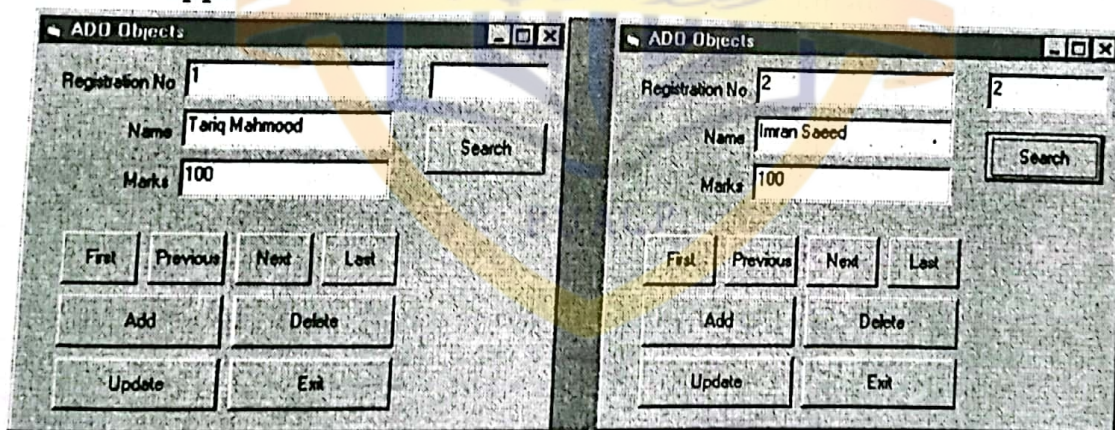
14. Add the following code in Click event of cmdExit:



15. Add the following code in Click event of cmdSearch:



16. Run the project. The main form will appear.
17. Enter 2 in the textbox above Search button and click Search. The search result will appear.



16.10 Microsoft FlexGrid Control

Microsoft Flex Grid is used to display data in tabular form. You can display a complete table in this control. It is normally used to display data that cannot be edited by the user:

16.10.1 Adding MSFlexGrid Control in Application

You need to add Microsoft FlexGrid control in the Visual Basic toolbox to use it. You can add it in the toolbox as follows:

1. Select Components from Project menu.
2. Select Microsoft FlexGrid Control 6.0 checkbox.
3. Click OK. The control will be added in the toolbox.

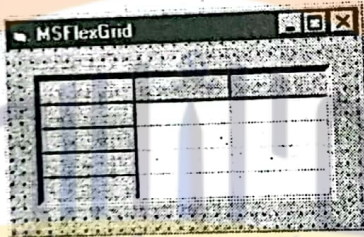
16.10.2 Important Properties of MSFlexGrid

Some important properties of MSFlexGrid control are as follows:

- **Row** – This property refers to the current row in the grid.
- **Col** – This property refers to the current column in the grid.
- **Rows** – This property is used to set the total number of rows in the grid.
- **Cols** – This property is used to set the total number of column in the grid.
- **Text** – This property refers to the text indicated by (Row, Col)
- **FixedCols** – This property is used to set the number of column for heading and is 1 by default.
- **FixedRows** – This property is used to set the number of rows for heading and is 1 by default.

Practical 16.6

1. Start a new Standard EXE project.
2. Add Microsoft FlexGrid control in the toolbox.
3. Double click MSFlexGrid control in the toolbox to add it on the form.
4. Set Rows property to 5.
5. Set Cols property to 3. The form should be displayed as follows:

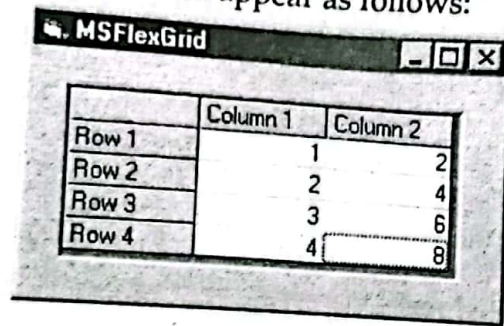


6. Add the following code in the Load event of the form.

```

Project1 - Form1 (Code)
Form Load
Private Sub Form_Load()
' Set column headings
MSFlexGrid1.Row = 0
For c = 1 To 2
    MSFlexGrid1.Col = c
    MSFlexGrid1.Text = "Column " & c
Next
' Set row headings
MSFlexGrid1.Col = 0
For r = 1 To 4
    MSFlexGrid1.Row = r
    MSFlexGrid1.Text = "Row " & r
Next
' Fill the grid
For r = 1 To 4
    MSFlexGrid1.Row = r
    For c = 1 To 2
        MSFlexGrid1.Col = c
        MSFlexGrid1.Text = r * c
    Next
Next
End Sub
  
```


7. Run the project. The form will appear as follows:



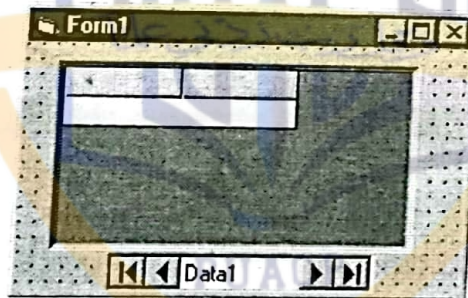
	Column 1	Column 2
Row 1	1	2
Row 2	2	4
Row 3	3	6
Row 4	4	8

16.10.3 Using MSFlexGrid with Database

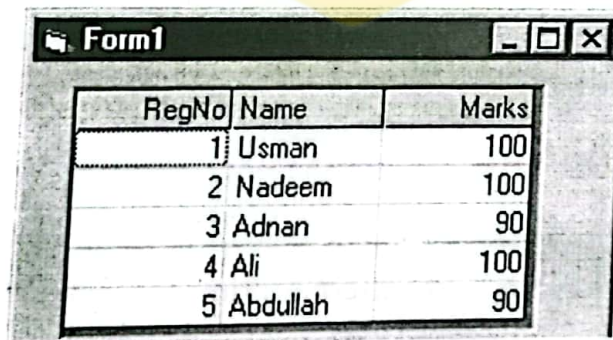
You can connect MSFlexGrid with a Data control to display the contents of a table. ADO Data control cannot be used with MSFlexGrid control.

Practical 16.7

1. Start a new Standard EXE project.
2. Add MSFlexGrid control in the toolbox from Components dialog box.
3. Add a Data Control on the form.
4. Add an MSFlexGrid control on the form.
5. Set DatabaseName property of Data Control to a database on your computer.
6. Select any table from RecordSource property of Data control.
7. Set Visible property of Data control to False.
8. Set DataSource property of MSFlexGrid control to Data1. The form should appear as follows:



9. Run the project. The form will appear as follows:



RegNo	Name	Marks
1	Usman	100
2	Nadeem	100
3	Adnan	90
4	Ali	100
5	Abdullah	90

16.11 Master-Detail Tables

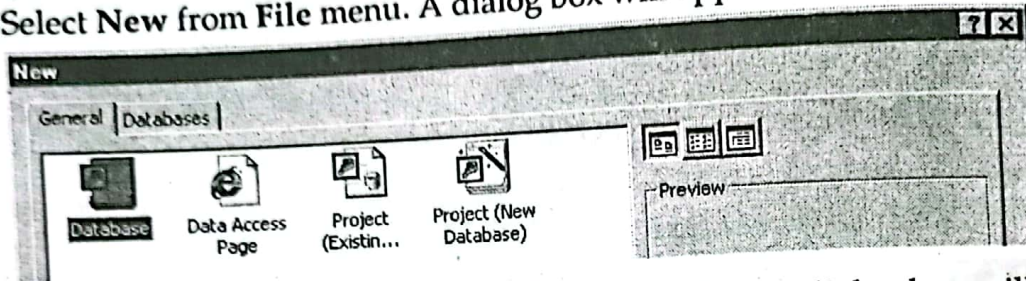
Sometimes, it is useful to store our data in two or more tables instead of only one. This allows us to utilize memory efficiently. The records are not stored repeatedly without any reason in the tables. When this situation occurs, you need to connect these tables by establishing a relationship between them.

16.11.1 Creating Master-Detail Tables in Microsoft Access

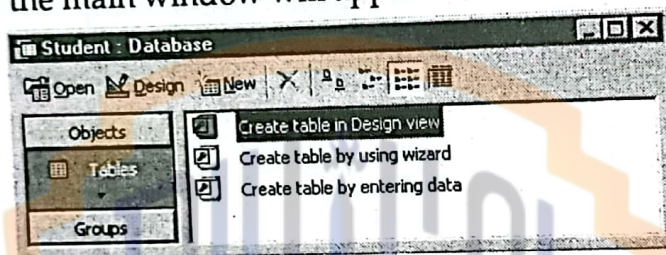
Microsoft Access allows us to create different tables and establish relationships between these tables.

Practical 16.8

1. Start Microsoft Access.
2. Select New from File menu. A dialog box will appear.



3. Choose Database option and click OK button. A new dialog box will appear.
4. Enter the name of the database and click Create button. A new database will be created and the main window will appear as follows:



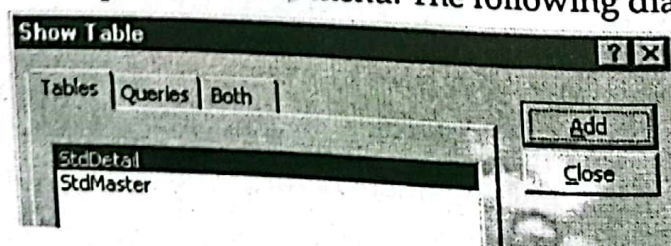
5. Double click on Create table in Design view option. The Table window will appear.
6. Create a table as follows:

	Field Name	Data Type	Description
PK	RegNo	Number	
	Name	Text	
	Address	Text	

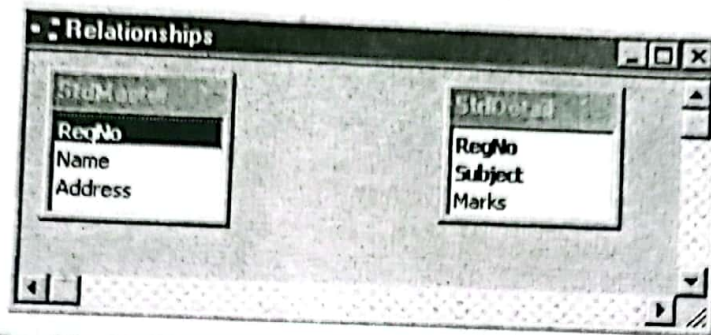
7. Set the field "RegNo" as Primary Key and save the table with "StdMaster" name.
8. Create the second tables with the following fields:

	Field Name	Data Type	Description
PK	RegNo	Number	
	Subject	Text	
	Marks	Number	

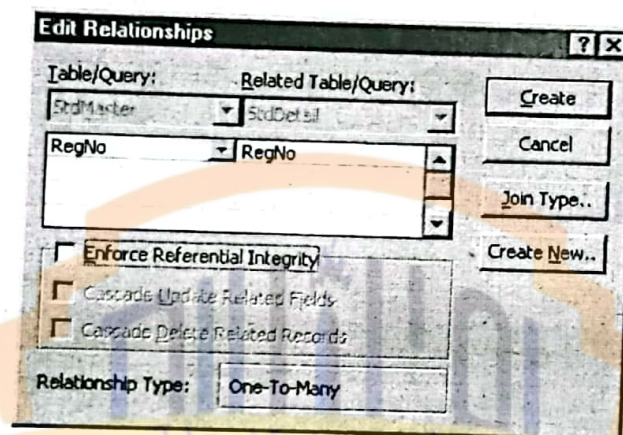
9. Set the fields RegNo and Subject as Primary Key and save table as StdDetail.
10. Select Relationships from Tools menu. The following dialog box will appear:



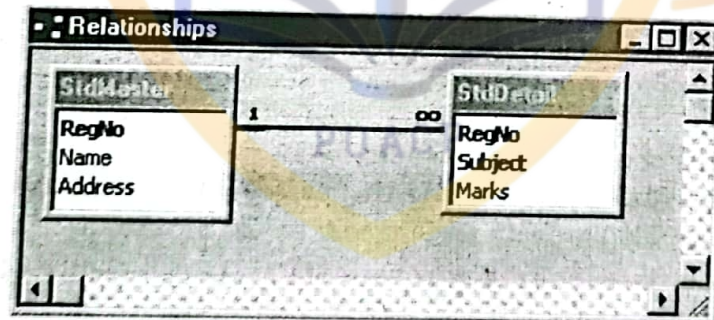
11. Select StdMaster table and click Add button.
12. Select StdDetail table and click Add button.
13. Click Close button to close dialog box. The Relationships window will appear as follows:



14. Click on RegNo in StdMaster table, drag it on the RegNo in StdDetail table and release the mouse. A dialog box will appear as follows:



15. Select **Enforce Referential Integrity** checkbox and click **Create** button. Both tables will be joined by one-to-many relationship as follows:



Here, StdMaster is the master table and StdDetail is the detail table. You cannot enter data in detail table without first entering the data of the same record in the master table. For example, if you need to enter the result of a student with RegNo 1, you have to first enter his record in the master table and then you can enter the detail of his marks in the detail table.

Note: The Data control does not work with the files of Microsoft 2000 or later. The database must be converted to prior version by using the following procedure:

- Select Tools > Database Utilities > Convert Database > To Access 97 File Format... A dialog box will appear to get new file name.
- Select a new location for the database and click Save button. The database will be converted to Access 97 format and saved at the new location.

16.11.2 Displaying Data from Master-Detail Tables

You can display data from master-detail tables. You will need two Data controls i.e. one for master table and one for detail table. It is easy to display the data of master table in simple controls like textboxes and the detail table in MSFlexGrid control.

Practical 16.9

1. Start a new Standard EXE project.
2. Place two Data control on the form.
3. Set the Caption property of Data1 to "Master".
4. Set the Caption property of Data2 to "Detail".
5. Set the Visible property of Data2 to False.
6. Set the DatabaseName property of Data1 to Students database and RecordSource property to StdMaster.
7. Set the DatabaseName property of Data2 to Students database and RecordSource property to StdDetail.
8. Place three labels and three textboxes on the form.
9. Set DataSource property of three textboxes to Data1.
10. Set DataField property of Text1 to RegNo.
11. Set DataField property of Text2 to Name.
12. Set DataField property of Text3 to Address.
13. Add the following code in the Reposition event of Data1 control:

```

Private Sub Data1_Reposition()
    Dim r As Integer
    r = Data1.Recordset!RegNo
    Data2.RecordSource = "Select * from StdDetail " & " Where RegNo = " & r
    Data2.Refresh
End Sub

```

14. Add an MSFlexGrid control in the form.
15. Set its Rows and Cols property to 3 and DataSource property to Data2.
16. Run the project. The data will appear in the forms.
17. Click on any navigational arrow of Data1. It will change both the master and the detail data as follows:

Registration No: 1

Name: Usman

Address: Faisalabad

Navigation: < < Master > >

RegNo	Subject	Marks
1	Computer	76
1	English	81
1	Math	90

Registration No: 2

Name: Adnan

Address: Faisalabad

Navigation: < < Master > >

RegNo	Subject	Marks
2	Computer	79
2	English	52
2	Math	37

Short Questions

Q.1. What is a DLL?

DLL stands for Dynamic Link Libraries. Windows uses DLLs to store collections of commonly used procedures. The functions available in Windows but not in Visual Basic can also be used. VB projects can call procedures from libraries. DLL file is linked to project when it runs. It is useful for procedures used by multiple projects.

Q.2. Explain the difference between a data control and a data-bound control.

Data control is created using intrinsic DataControl tool from toolbox. It generally links one form with one table. Data-bound controls such as text boxes and labels display the actual data from fields in the attached database table.

Q.3. How are Seek and FindFirst methods similar or different?

Both methods find the first record that satisfies the given criteria. Seek method works with table-type recordsets but FindFirst works with snapshot and dynaset-type recordsets.

Q.4. What is difference between FindFirst and MoveFirst methods?

MoveFirst method positions the record pointer on first record whereas FindFirst method positions the record pointer on first record that meets stated criteria.

Q.5. How BOF and EOF properties are set used in a project?

BOF property is automatically set to True when the record pointer is before the first record in Recordset. It happens when the first record is current and the user chooses MovePrevious. BOF property is also true if the Recordset is empty.

EOF property is True when the record pointer moves beyond the last record in Recordset and when the Recordset is empty. While writing code for navigation, it is important to check BOF and EOF so that run-time errors do not occur.

Q.6. What steps are needed to change the data in a database record?

Recordset object has an Update method that is used to save any changes in the data. Visual Basic automatically executes Update method any time the user clicks one of the navigation buttons or executes any Move method after making any change.

Q.7. Explain the differences between Find and Filter properties of a Recordset.

Find method searches the current recordset for the first record that matches the specified criteria. Filter property creates a temporary recordset of all records that match the specified criteria.

Q.8. Specify the methods used to navigate through the Recordset.

The methods used for navigating through recordset are MoveFirst, MoveNext, MovePrevious and MoveLast.

Q.9. Name the three different VB database object types.

Visual Basic contains three different data object types: Data Access Objects (DAO), Remote Data Objects (RDO) and ActiveX Data Objects (ADO). ActiveX Data Objects (ADO) are new to VB 6.0 and are used to access a database on a local computer system, through a network or over the Internet.

Multiple Choice

- The intermediary between application and DBMS drivers in ODBC architecture is:
 - Driver manager
 - OLE DB interface
 - ODBC driver
 - Data source
- Which of the following is a set of functions that the application can call to receive services?
 - OLE Object
 - DBMS data Source
 - API
 - System

3. A Command object is used to:
 - a. Get records from a database.
 - b. Execute SQL queries
 - c. Directly manipulate data.
 - d. All
4. Which of the following property indicates the current row of MSFlexGrid?
 - a. CurrentRow
 - b. intCurrentRow
 - c. Row
 - d. FixedRows
5. Which of the following property indicates the total number rows in a MSFlexGrid?
 - a. Row
 - b. TotalRow
 - c. intTotalRows
 - d. FixedRows
6. Which of the following property indicates the current column of MSFlexGrid?
 - a. Cols
 - b. CurrentCol
 - c. intCurrentCol
 - d. FixedCols
7. Which of the following property indicates the total number rows in MSFlexGrid?
 - a. FixedCols
 - b. TotalCol
 - c. intTotalCols
 - d. Col
8. In transaction processing, ending current transaction and saving the changes is initiated by:
 - a. Rollback
 - b. SaveChanges
 - c. SaveTrans
 - d. CommitTrans
9. In a multi-user environment, the record locking method that only locks the record while changes are written to the database is called :
 - a. Optimistic locking
 - b. Pessimistic locking
 - c. Edit locking
 - d. Change locking
10. In a multi-user environment, which locking method prevents all other users from updating a record while the current user is editing the record?
 - a. Pessimistic
 - b. Optimistic
 - c. Shared
 - d. Read-only
11. Which method is used to begin a new transaction?
 - a. StartTrans
 - b. NewTrans
 - c. BeginTrans
 - d. InitTrans
12. Which method positions the record pointer on first record of table-type recordset that meets a stated condition?
 - a. Seek
 - b. Find
 - c. MoveFirst
 - d. FindFirst
13. To move the record pointer to the next record, which method should be called?
 - a. FindFirst
 - b. FindNext
 - c. MoveFirst
 - d. MoveNext
14. Methods that position a recordset's pointer include:
 - a. Move
 - b. MoveLast
 - c. MoveFirst
 - d. All

Answers

1. a	2. c	3. d	4. c
5. d	6. a	7. a	8. d
9. a	10. a	11. c	12. a
13. d	14. d	15.	16.

Fill in the Blanks

1. The _____ makes it possible to access both local and remote databases.
2. Three main objects of ADO object model required to access data from database are _____ and _____.
3. The _____ method cancels the changes before they are saved to the data source.
4. The process of attaching a data-aware control to a data control is called _____.

5. _____ provides an interface to OLE DB.
6. Controls can be bound to ADO Data control if they possess _____.
7. _____ is an alternative to creating database application through code.
8. The _____ object creates a connection between an application and the data source.
9. The _____ object is used to access records returned from a query.
10. _____ object is used to query a database and access the records returned from database.
11. To create a Connection object we need to set _____ and _____ properties.
12. To display data from a Recordset, we use _____ operator or the Fields collection.
13. Four primary methods to navigate Recordset are _____.
14. The _____ method is used to insert a record into a Recordset.
15. The _____ method deletes the current record from a Recordset.
16. We use _____ to execute stored procedures and SQL commands that have parameters.

Answers

1. ADODC	2. Connection, Recordset and Command
3. CancelUpdate	4. Binding
5. ActiveX Data Object(ADO)	6. DataSource property
7. ActiveX Data Object(ADO	8. Connection
9. Recordset	10. Command
11. ConnectionString, Connection Timeout	12. Bang(')
13. MoveNext, MovePrevious, MoveFirst, MoveLast	14. addnew
15. Delete	16. Command Object

True / False

1. It is possible to build application that is able to communicate with different databases.
2. ODBC uses a standard method for communication between application and DBMS.
3. ActiveX Data Object is a Microsoft technology.
4. ConnectionString is a method of ADO data control.
5. Command object is used to establish connection between data source and VB application.
6. ADO Recordset object is used to store the data retrieved from a database.
7. EOF property returns Boolean value indicating if the current record position is before the first record or not.
8. Close, delete and find are properties of Recordset object.
9. Row property of MSFlexGrid refers to the current rows in the grid.
10. Microsoft FlexGrid is used to display data in tabular form.

Answers

1. T	2. T	3. T	4. F
5. F	6. T	7. F	8. F
9. T	10. T	11.	12.

Project 10

Address Book

Project Specification

The project Address Book uses the following forms:

- **frmLogon:** It is used to input user and password from the user to login.
- **MDIForm1:** It is used as parent window. It displays different menus.
- **frmNewUser:** It is used to create a new user.
- **frmEditPassword:** It is used to edit the password of an existing user.
- **frmBackup:** It is used to take backup of the database.
- **frmRestore:** It is used to restore a database.
- **frmSearch:** It is used to get search criteria from the user.
- **frmMain:** It is used to display and manipulate data from the database.
- **frmSearchResult:** It is used to display the result of search.

The project uses a module **modMain** that contains public variables and two sub procedures.

Procedure

Design Database

1. Start MS Access and creates a new database **Address**.
2. Create a new table **Users** with the following fields:

Users : Table			
	Field Name	Data Type	Description
	UserName	Text	Primary Key, User of the software. Set Field size to 25.
	UPassword	Text	Set Field size to 8.

3. Set **UserName** field as primary key.
4. Add at least one username and password in the table.
5. Create a new table **Contact** with the following fields:

Contact : Table			
	Field Name	Data Type	Description
	ID	Text	Primary key, Unique ID of each record.
	Name	Text	Set Field size to 50.
	Address	Text	Set Field size to 100.
	City	Text	Set Field size to 30.
	Country	Text	Set Field size to 20.
	Phone	Text	Set Field size to 15.
	Email	Text	Set Field size to 25.

6. Set **ID** field as primary key.

Main Form

1. Start a new Standard EXE project in Visual Basic.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmMain	Caption	Main Form
		MDIChild	True
Frame	frmAddress	Caption	Address Book
Label	lblID	Caption	ID
Label	lblName	Caption	Name
Label	lblAddress	Caption	Address
Label	lblCity	Caption	City
Label	lblCountry	Caption	Country
Label	lblPhone	Caption	Phone
Label	lblEmail	Caption	Email
Textbox	txtID	Text	Blank
Textbox	txtName	Text	Blank
Textbox	txtAddress	Text	Blank
Textbox	txtCity	Text	Blank
Textbox	txtCountry	Text	Blank
Textbox	txtPhone	Text	Blank
Textbox	txtEmail	Text	Blank
CommandButton	cmdFirst	Caption	<<
CommandButton	cmdPrevious	Caption	<
CommandButton	cmdNext	Caption	>
CommandButton	cmdLast	Caption	>>
CommandButton	cmdAdd	Caption	Add
CommandButton	cmdSave	Caption	Save
CommandButton	cmdDelete	Caption	Delete
CommandButton	cmdUpdate	Caption	Update
CommandButton	cmdClose	Caption	Close

3. Design the form as follows:

The screenshot shows a Windows-style application window titled 'Main Form'. Inside the window is a sub-form titled 'Address Book'. The sub-form has seven labeled input fields: ID, Name, Address, City, Country, Phone, and Email. Below these fields is a row of five buttons: '<<', '<', '>', '>>', and 'Close'. Below that row is another row of four buttons: 'Add', 'Save', 'Delete', and 'Update'.

4. Add the following code in Click event of **cmdFirst** button:

```
Private Sub cmdFirst_Click()
r.MoveFirst
Call Fill
End Sub
```

5. Add the following code in Click event of **cmdPrevious** button:

```
Private Sub cmdPrevious_Click()
r.MovePrevious
If r.BOF Then
    r.MoveFirst
End If
Call Fill
End Sub
```

6. Add the following code in Click event of **cmdNext** button:

```
Private Sub cmdNext_Click()
r.MoveNext
If r.EOF Then
    r.MoveLast
End If
Call Fill
End Sub
```

7. Add the following code in Click event of **cmdLast** button:

```
Private Sub cmdLast_Click()
r.MoveLast
Call Fill
End Sub
```

8. Add the following code in Click event of **cmdAdd** button:

```
Private Sub cmdAdd_Click()
Call FillID
txtName.Text = ""
txtAddress.Text = ""
txtCity.Text = ""
txtCountry.Text = ""
txtPhone.Text = ""
txtEmail.Text = ""
txtName.SetFocus
End Sub
```

9. Add the following code in Click event of **cmdSave** button:

```
Private Sub cmdSave_Click()
On Error GoTo ErrHand
If txtName.Text = "" Then
    MsgBox "Enter Name!", vbCritical, "Save"
    txtName.SetFocus
ElseIf txtAddress.Text = "" Then
    MsgBox "Enter Address!", vbCritical, "Save"
    txtAddress.SetFocus
```



```

ElseIf txtCity.Text = "" Then
    MsgBox "Enter Phone!", vbCritical, "Save"
    txtCity.SetFocus
ElseIf txtCountry.Text = "" Then
    MsgBox "Enter Phone!", vbCritical, "Save"
    txtCountry.SetFocus
ElseIf txtPhone.Text = "" Then
    MsgBox "Enter Phone!", vbCritical, "Save"
    txtPhone.SetFocus
ElseIf txtEmail.Text = "" Then
    MsgBox "Enter Email!", vbCritical, "Save"
    txtEmail.SetFocus
Else
    Set rCheck = c.Execute("Select * from Contact where Email=" & _
        txtEmail.Text & ";")
    If rCheck.EOF Then
        c.Execute "Insert into Contact(ID, Name, Address, City, Country, Phone,Email) values(" & _
            & txtID.Text & "," & txtName.Text & "," & txtAddress.Text & "," & txtCity.Text & "," & _
            txtCountry.Text & "," & txtPhone.Text & "," & txtEmail.Text & ");"
        MsgBox "Record saved successfully!", vbInformation, "Save"
        r.Requery
        Call Fill
    Else
        MsgBox "Record already exists!"
    End If
End If
Exit Sub
ErrHand:
    MsgBox Err.Description, vbCritical, "Error"

```

10. Add the following code in Click event of **cmdDelete** button:

```

c.Execute ("Delete from Contact where ID = " & txtID.Text & ";")
r.Requery
Call Fill
MsgBox "Record deleted sucessfully!"
End Sub

```

11. Add the following code in Click event of **cmdUpdate** button:

```

Private Sub cmdUpdate_Click()
On Error GoTo ErrHand
If txtName.Text = "" Then
    MsgBox "Enter Name!", vbCritical, "Save"
    txtName.SetFocus
ElseIf txtAddress.Text = "" Then
    MsgBox "Enter Address!", vbCritical, "Save"
    txtAddress.SetFocus
ElseIf txtPhone.Text = "" Then
    MsgBox "Enter Phone!", vbCritical, "Save"
    txtPhone.SetFocus
ElseIf txtEmail.Text = "" Then
    MsgBox "Enter Email!", vbCritical, "Save"
    txtEmail.SetFocus

```


Else

```
c.Execute ("Update Contact set Name=" & txtName.Text & ",Address=" & txtAddress.Text & ",City=" & txtCity.Text & ",Country=" & txtCountry.Text & ",Phone=" & txtPhone.Text & ",Email=" & txtEmail.Text & " where ID=" & txtID.Text & ";")
```

```
MsgBox "Record updated successfully!", vbInformation, "Save"
```

End If

Exit Sub

ErrHand:

```
MsgBox Err.Description, vbCritical, "Error"
```

End Sub

12. Add the following code in Click event of cmdClose button:

```
Private Sub cmdClose_Click()
```

```
Unload Me
```

```
End Sub
```

Adding Module

1. Select **Project > Add Module** to add a new module in the project.

2. Declare the following variables in the module code window:

```
Public c As New ADODB.Connection
```

```
Public r As New ADODB.Recordset
```

```
Public rSearch As New ADODB.Recordset
```

```
Public rCheck As New ADODB.Recordset
```

```
Public s As String
```

```
Public search As String
```

The purpose of each variables declared above is as follows:

Variable	Purpose
c	An object variable to establish a connection with the database.
r	A recordset object to retrieve data from the database.
rSearch	A recordset object to retrieve data from the database.
rCheck	A recordset object to retrieve data from the database.
s	A string variable to store information about the connection object.
search	A string variable to store the value of different search criteria.

3. Declare the following sub procedure in the module:

```
Public Sub Connect()
```

```
s = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=" & _  
App.Path & "Address.mdb"
```

```
c.Open s
```

```
r.Open "Contact", c, adOpenDynamic, adLockOptimistic
```

```
End Sub
```

The above sub procedure is used to establish a connection with Address database.

4. Declare the following sub procedure in the module:

```
Public Sub CenterForm(frmTemp As Form)
```

```
On Error Resume Next
```

```
frmTemp.Left = (Screen.Width - frmTemp.Width) / 2
```

```
frmTemp.Top = (Screen.Height - frmTemp.Height) / 2 - 500
```

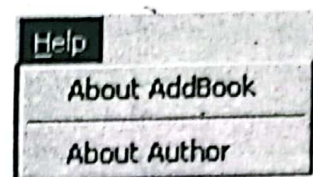
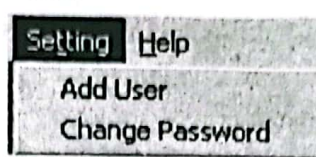
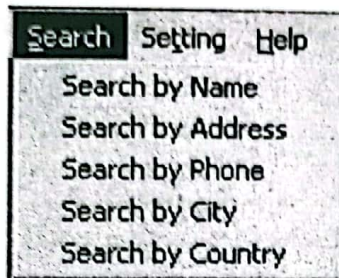
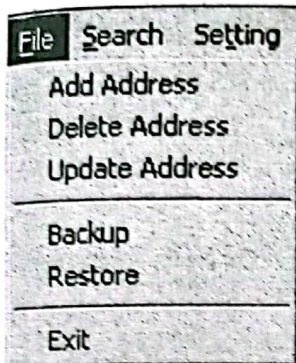
```
Exit Sub
```

```
End Sub
```


The above sub procedure is used to open each form in the center of the screen. It accepts a form as parameter and displays it in the center.

MDI Form

1. Select **Project > Add MDI Form**.
2. Add the following menus in the MDI Form.



3. Add the following code in Click event of **Add Address** menu:

```
Private Sub mnuAdd_Click()  
Form1.Show  
End Sub
```

4. Add the following code in Click event of **Delete Address** menu:

```
Private Sub mnuDelete_Click()  
Form1.Show  
End Sub
```

5. Add the following code in Click event of **Update Address** menu:

```
Private Sub mnuUpdate_Click()  
Form1.Show  
End Sub
```

6. Add the following code in Click event of **Backup** menu:

```
Private Sub mnuBackup_Click()  
frmBackup.Show  
End Sub
```

7. Add the following code in Click event of **Restore** menu:

```
Private Sub mnuRestore_Click()  
frmRestore.Show  
End Sub
```

8. Add the following code in Click event of **Exit** menu:

```
Private Sub exit_Click()  
If MsgBox("Sure to exit?", vbYesNo, "Exit") = vbYes Then  
End  
End If  
End Sub
```

9. Add the following code in Click event of **Search by Name** menu:

```
Private Sub mnuByName_Click()  
modMain.search = "Name"  
frmSearch.Show  
End Sub
```


10. Add the following code in Click event of Search by Address menu:

```
Private Sub mnuByAddress_Click()  
modMain.search = "Address"  
frmSearch.Show  
End Sub
```

11. Add the following code in Click event of Search by Phone menu:

```
Private Sub mnuByPhone_Click()  
modMain.search = "Phone"  
frmSearch.Show  
End Sub
```

12. Add the following code in Click event of Search by City menu:

```
Private Sub mnuByCity_Click()  
modMain.search = "City"  
frmSearch.Show  
End Sub
```

13. Add the following code in Click event of Search by Country menu:

```
Private Sub mnuByCountry_Click()  
modMain.search = "Country"  
frmSearch.Show  
End Sub
```

14. Add the following code in Click event of Add User menu:

```
Private Sub mnuAddUser_Click()  
frmNewUser.Show  
End Sub
```

15. Add the following code in Click event of Search by Name menu:

```
Private Sub mnuByName_Click()  
modMain.search = "Name"  
frmSearch.Show  
End Sub
```

16. Add the following code in Click event of Search by Name menu:

```
Private Sub mnuByName_Click()  
modMain.search = "Name"  
frmSearch.Show  
End Sub
```

17. Add the following code in Click event of Search by Name menu:

```
Private Sub mnuByName_Click()  
modMain.search = "Name"  
frmSearch.Show  
End Sub
```

18. Add the following code in Click event of Add User menu:

```
Private Sub adduser_Click()  
frmNewUser.Show  
End Sub
```

19. Add the following code in Click event of Change Password menu:

```
Private Sub changepass_Click()
```



```
frmEditUserPassword.Show
End Sub
```

20. Add the following code in Click event of **About AddBook** menu:

```
Private Sub about_Click()
MsgBox "Address Book from IT Series...", vbInformation, "Address Book"
End Sub
```

21. Add the following code in Click event of **About Author** menu:

```
Private Sub author_Click()
MsgBox "Designed & Developer by" & vbCrLf & "Tariq Mahmood" & _
vbCrLf & "Imran Saeed" & vbCrLf & "Tasleem Mustafa" & vbCrLf & "Ahsan Raza", , "Project"
End Sub
```

Login Form

22. Select **Project > Add Form**. A new form will be added in the form.

23. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmLogon	Caption	Address Book
Label	lblMsg	Caption	Please enter your name and password.
Label	lblName	Caption	Login Name
Label	lblPassword	Caption	Password
Textbox	txtName	Text	Blank
Textbox	txtPassword	Text	Blank
CommandButton	cmdOK	Caption	OK
CommandButton	cmdExit	Caption	Exit

24. Design the form as follows:

25. Add the following code in **General Declaration** section:

```
Option Explicit
Dim rs As New ADODB.Recordset
```

26. Add the following code in **Load** event of the form:

```
Private Sub Form_Load()
Call CenterForm(Me)
Call Connect
End Sub
```

27. Add the following code in Click event of **cmdOK** button:

```
Private Sub cmdOk_Click()
If rs.State <> 1 Then
rs.Open "Users", c, adOpenDynamic, adLockOptimistic, adCmdTable
End If
```



```

rs.MoveFirst
Do While Not rs.EOF
    If txtLoginName.Text = rs.Fields("UserName").Value And txtPassword.Text =
rs.Fields("UPassword") Then
        Me.Hide
        MDIForm1.Show
        Exit Sub
    End If
rs.MoveNext
Loop
If rs.EOF Then
    MsgBox "Invalid username or password!"
End If
rs.Close
End Sub

```

28. Add the following code in Click event of cmdExit button:

```

Private Sub cmdExit_Click()
    End
End Sub

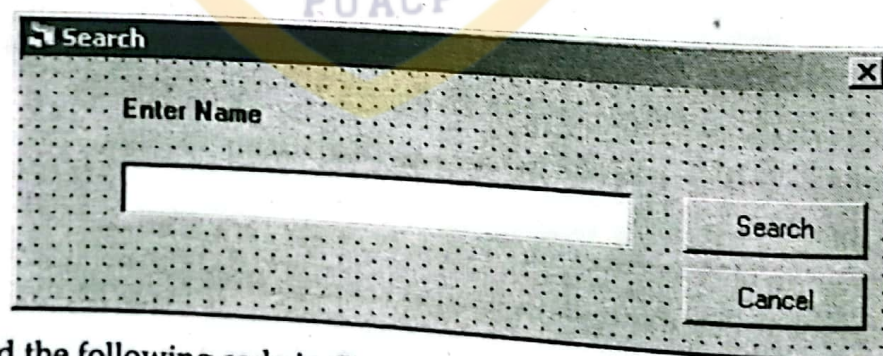
```

Search Criteria Form

1. Select Project > Add Form. A new form will be added in the form.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmSearch	Caption	Search
		MDIChild	True
Label	lblName	Caption	Enter Name
Textbox	txtCriteria	Text	Blank
CommandButton	cmdSearch	Caption	Search
CommandButton	cmdCancel	Caption	Cancel

3. Design the form as follows:



4. Add the following code in General Declaration section:

Option Explicit

5. Add the following code in Load event of the form:

```

Private Sub Form_Load()
    lblName.Caption = "Enter " & search
    Call modMain.CenterForm(Me)
End Sub

```


6. Add the following code in Click event of Search button:

```
Private Sub cmdSearch_Click()
Dim Criteria As String
Criteria = txtCriteria.Text
If search = "Name" Then
    rSearch.Open "Select * from Contact where Name=" & Criteria & "", c, adOpenDynamic,
adLockOptimistic
ElseIf search = "Address" Then
    rSearch.Open "Select * from Contact where Address=" & Criteria & "", c, adOpenDynamic,
adLockOptimistic
ElseIf search = "City" Then
    rSearch.Open "Select * from Contact where City=" & Criteria & "", c, adOpenDynamic,
adLockOptimistic
ElseIf search = "Country" Then
    rSearch.Open "Select * from Contact where Country=" & Criteria & "", c, adOpenDynamic,
adLockOptimistic
ElseIf search = "Phone" Then
    rSearch.Open "Select * from Contact where Phone=" & Criteria & "", c, adOpenDynamic,
adLockOptimistic
ElseIf search = "Email" Then
    rSearch.Open "Select * from Contact where Email=" & Criteria & "", c, adOpenDynamic,
adLockOptimistic
End If
If rSearch.EOF Then
    MsgBox "No Record found!"
    rSearch.Close
Else
    frmSearchResult.Show
    Unload Me
End If
End Sub
```

7. Add the following code in Click event of Cancel button:

```
Private Sub cmdCancel_Click()
Unload Me
End Sub
```

Search Result Form

1. Select **Project > Add Form**. A new form will be added in the form.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmSearchResult	Caption	Search Result
		MDIChild	True
Frame	frmAddress	Caption	Address Book
Label	lblID	Caption	ID
Label	lblName	Caption	Name
Label	lblAddress	Caption	Address
Label	lblCity	Caption	City
Label	lblCountry	Caption	Country

Label	lblPhone	Caption	Phone
Label	lblEmail	Caption	Email
Textbox	txtID	Text	Blank
Textbox	txtName	Text	Blank
Textbox	txtAddress	Text	Blank
Textbox	txtCity	Text	Blank
Textbox	txtCountry	Text	Blank
Textbox	txtPhone	Text	Blank
Textbox	txtEmail	Text	Blank
CommandButton	cmdFirst	Caption	<<
CommandButton	cmdPrevious	Caption	<
CommandButton	cmdNext	Caption	>
CommandButton	cmdLast	Caption	>>
CommandButton	cmdClose	Caption	Close

3. Design the form as follows:

4. Add the following code in Click event of cmdFirst button:

```
Private Sub cmdFirst_Click()
    rSearch.MoveFirst
    Call Fill
End Sub
```

5. Add the following code in Click event of cmdPrevious button:

```
Private Sub cmdPrevious_Click()
    rSearch.MovePrevious
    If rSearch.BOF Then
        rSearch.MoveFirst
    End If
    Call Fill
End Sub
```


6. Add the following code in Click event of cmdNext button:

```
Private Sub cmdNext_Click()
rSearch.MoveNext *
If rSearch.EOF Then
    rSearch.MoveLast
End If
Call Fill
End Sub
```

7. Add the following code in Click event of cmdLast button:

```
Private Sub cmdLast_Click()
rSearch.MoveLast
Call Fill
End Sub
```

8. Add the following code in Click event of Close button:

```
Private Sub cmdCancel_Click()
Unload Me
End Sub
```

9. Declare the following sub procedure in the code windows of the form:

```
Sub Fill()
txtID.Text = rSearch("ID")
txtName.Text = rSearch("Name")
txtAddress.Text = rSearch("Address")
txtCity.Text = rSearch("City")
txtCountry.Text = rSearch("Country")
txtPhone.Text = rSearch("Phone")
txtEmail.Text = rSearch("Email")
End Sub
```

10. Add the following code in Unload event of the form:

```
Private Sub Form_Unload(Cancel As Integer)
rSearch.Close
End Sub
```

Backup Data Form

1. Select Project > Add Form. A new form will be added in the form.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmBackup	Caption	Backup Data
		MDIChild	True
Label	lblPath	Caption	Path
Textbox	txtPath	Text	Blank
CommandButton	cmdBrowse	Caption	Browse
CommandButton	cmdBackup	Caption	Backup
CommandButton	cmdClose	Caption	Close
CommonDialog	ComDlg	CancelError	False

3. Design the form as follows:

4. Add the following code in **General Declaration** section of the form:

Option Explicit

Dim Files As New FileSystemObject

5. Add the following code in **Load** event of the form:

```
Private Sub Form_Load()
```

```
Call CenterForm(Me)
```

```
txtPath.Text = App.Path & "\Address.mdb"
```

```
End Sub
```

6. Add the following code in **Click** event of **Browse** button:

```
Private Sub cmdBrowse_Click()
```

```
ComDlg.Filter = " Microsoft Access Files (*.mdb)|*.mdb"
```

```
ComDlg.FilterIndex = 1
```

```
ComDlg.ShowSave
```

```
txtpath.Text = ComDlg.FileName
```

```
Exit Sub
```

```
End Sub
```

7. Add the following code in **Click** event of **Backup** button:

```
Private Sub cmdBackup_Click()
```

```
On Error GoTo ErrEnd
```

```
If Trim(txtpath.Text) = Empty Then
```

```
MsgBox "Enter Valid Path", vbInformation, "Backup"
```

```
Exit Sub
```

```
End If
```

```
If Files.FileExists(txtpath.Text) = True Then
```

```
If txtpath.Text = App.Path & "\Address.mdb" Then
```

```
MsgBox "Enter Valid Path", vbInformation, "Backup"
```

```
Exit Sub
```

```
End If
```

```
Files.DeleteFile (txtpath.Text)
```

```
Files.CopyFile App.Path & "\Address.mdb", txtpath.Text, True
```

```
Else
```

```
Files.CopyFile App.Path & "\Address.mdb", txtpath.Text, True
```

```
End If
```

```
MsgBox "Data Backup Successfully ", vbInformation, "Backup"
```

```
Exit Sub
```

```
ErrEnd:
```

```
MsgBox "Back unsucessfull due to " & Err.Description
```

```
End Sub
```


8. Add the following code in Click event of Close button;

```
Private Sub cmdCancel_Click()
Unload Me
End Sub
```

Restore Data Form

1. Select Project > Add Form. A new form will be added in the form.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmRestore	Caption	Restore Data
		MDIChild	True
Label	lblPath	Caption	Path
Textbox	txtPath	Text	Blank
CommandButton	cmdBrowse	Caption	Browse
CommandButton	cmdRestore	Caption	Restore
CommandButton	cmdClose	Caption	Close
CommonDialog	ComDlg	CancelError	False

3. Design the form as follows:

4. Add the following code in General Declaration section of the form:

```
Option Explicit
Dim Files As New FileSystemObject
```

5. Add the following code in Load event of the form:

```
Private Sub Form_Load()
Call CenterForm(Me)
txtPath.Text = App.Path & "\Address.mdb"
End Sub
```

6. Add the following code in Click event of Browse button:

```
Private Sub cmdBrowse_Click()
ComDlg.Filter = "Microsoft Access Files (*.mdb)|*.mdb"
ComDlg.FilterIndex = 1
ComDlg.ShowSave
txtPath.Text = ComDlg.FileName
Exit Sub
End Sub
```

7. Add the following code in Click event of Restore button:

```
Private Sub cmdRestore_Click()
On Error GoTo ErrEnd
```

```

If Trim(txtpath.Text) = Empty Then
    MsgBox "Enter Valid Path", vbInformation, "Restore"
    Exit Sub
End If
If Files.FileExists(txtpath.Text) = True Then
    If txtpath.Text = App.Path & "\Address.mdb" Then
        MsgBox "Enter Valid Path", vbInformation, "Restore"
        Exit Sub
    End If
    cn.Close
    Files.CopyFile txtpath.Text, App.Path & "\Address.mdb", True
    cn.Open
Else
    MsgBox "Enter Valid Path", vbInformation, "Restore"
    Exit Sub
End If
MsgBox "Data Restored Successfully ", vbInformation, "Restore"
Exit Sub
ErrEnd:
    MsgBox "Restore unsuccessful.", vbInformation
End Sub

```

8. Add the following code in Click event of Close button:

```

Private Sub cmdCancel_Click()
    Unload Me
End Sub

```

New User Form

1. Select Project > Add Form. A new form will be added in the form.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmNewUser	Caption	New User
		MDIChild	True
Label	lblName	Caption	Login Name
Label	lblPassword	Caption	Password
Textbox	txtName	Text	Blank
Textbox	txtPassword	Text	Blank
		PasswordChar	*
CommandButton	cmdSave	Caption	Save
CommandButton	cmdCancel	Caption	Cancel

3. Design the form as follows:

4. Add the following code in **General Declaration** section of the form:

Option Explicit

5. Add the following code in **Load** event of the form:

```
Private Sub Form_Load()
    Call CenterForm(Me)
End Sub
```

6. Add the following code in **Click** event of **Savebutton**:

```
Private Sub cmdSave_Click()
    Dim rs As New ADODB.Recordset
    If txtLoginName.Text = "" Then
        MsgBox "Enter the User Name"
    Else
        If r.State <> 1 Then
            r.Open "Users", c, adOpenDynamic, adLockOptimistic, adCmdTable
        End If
        r.MoveFirst
        Do While Not r.EOF
            If txtLoginName.Text = r.Fields("UserName") Then
                MsgBox "User Already Exist"
                r.Close
                Exit Sub
            End If
            r.MoveNext
        Loop
        r.AddNew
        r.Fields("UserName").Value = txtLoginName.Text
        r.Fields("UPassword").Value = txtPassword.Text
        r.update
        r.Close
        MsgBox "New user created!"
    End If
    txtLoginName.Text = ""
    txtPassword.Text = ""
End Sub
```

7. Add the following code in **Click** event of **Cancel** button:

```
Private Sub cmdCancel_Click()
    Unload Me
End Sub
```

Change Password Form

1. Select **Project > Add Form**. A new form will be added in the form.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Form	frmEditPassword	Caption	Change Password
		MDIChild	True

Label	lblName	Caption	Login Name
Label	lblOldPass	Caption	Old Password
Label	lblNewPass1	Caption	New Password
Label	lblNewPass2	Caption	Confirm Password
Textbox	txtName	Text	Blank
Textbox	txtPassword	Text	Blank
		PasswordChar	*
Textbox	txtNewPass1	Text	Blank
		PasswordChar	*
Textbox	txtNewPass2	Text	Blank
		PasswordChar	*
CommandButton	cmdSave	Caption	Save
CommandButton	cmdCancel	Caption	Cancel

3. Design the form as follows:

4. Add the following code in **General Declaration** section of the form:

Option Explicit

5. Add the following code in **Load** event of the form:

```
Private Sub Form_Load()
    Call CenterForm(Me)
End Sub
```

6. Add the following code in **Click** event of **Save** button:

```
Private Sub cmdSave_Click()
    Dim rs As New ADODB.Recordset
    If rs.State = 1 Then
        rs.Close
    End If
    If rs.State <> 1 Then
        rs.Open "Users", c, adOpenDynamic, adLockOptimistic, adCmdTable
    End If
    rs.MoveFirst
    Do While Not rs.EOF
        If rs.Fields("UserName").Value = txtLoginName.Text And rs.Fields("UPassword").Value =
        txtOldPassword.Text And txtNew.Text = txtConfirm.Text Then
            rs.Fields("UPassword").Value = txtNew.Text
        End If
    Loop
End Sub
```



```
rs.update
rs.Close
MsgBox "Password Changed!"
Unload Me
Exit Sub
End If
rs.MoveNext

Loop
If rs.EOF Then
MsgBox "Invalid Old Password!"
End If
rs.Close
End Sub
```

7. Add the following code in Click event of Cancel button:

```
Private Sub cmdCancel_Click()
Unload Me
End Sub
```

8. Run the project and check its working.

